

IN3026 - Project Report

Abhijit Shidlagatta – 210012420

Overview

As mentioned in my original milestone 1 project definition my goal was to create a game that had similar mechanics to a game such as Dark Souls or Elden Ring. I feel that I have successfully created a Souls-like game for coursework project.

Essentially it is a souls like game that focuses on stamina management, soul collecting, upgrading stats and large scale modelling and souls like combat.

Below is a list of all the specifications I have met according to requirements from the mark scheme and extras:

- Category: Game Modelling:
 - Intro Screen with controls- I have implemented a GameState::MainMenu with keyboard controls and a navigation system to allow the player to choose between options such as Start, Sensitivity (for camera) and Volume (for background music) (See Appendix 6 for main menu screenshot)
 - Objects using primitives- I was able to incorporate 5 unique primitives being the tetrahedron(Rock mound for the halberd), octahedron(Soul Fragments), Cuboid(Stone Block pillar to the left of spawn), Cube(Main castle wall), Cuboids(2 for the castle towers) (See **Appendix 3** for proof of making an octahedron which is the Soul Fragment)
 - New skybox and terrain- I achieved this in the first milestone already, so I didn't need to add onto this but yes this is achieved too
 - Audio- I used 4 synchronised sounds being the looping background music, attack slash sound effect, dodge roll sound effect and soul pickup
- Category: Camera, meshes, FX, Lighting:
 - Camera-motion technique- I provided a 3rd person camera that orbits the player, and this time took the milestone 1 feedback to also clamp the camera when it attempts to go below the floor. This way it will never see below the floor. Otherwise, it works as intended
 - Head's up display (HUD)- I implemented a working HUD with HP, Stamina, Potion count, Soul count and a Health Bar for the main boss of my Souls like game. In addition, I was able to clamp the health bars to each individual enemy after a lot of effort.
 - Usage of lighting and colouring effects- I used 5 point lights being the red light at the massive silver halberd statue and 4 yellow lights for the trees to allow it to seem like a sunset environment with a few fireflies around trees emitting faint light. The directional light is also present but is set to a low level to mimic the sunset and the skybox I chose.
 - Special effects used- I used transparency for the soul fragment primitive and used a method to render a random assortment of trees around a circle and a random assortment of grass all around the map with different rotations too. (See **Appendix 7** for an in-game image of the projectile)

- Category: Physics, AI and gameplay:
 - Use of game physics- I used bullet rigid body for all of my environment collisions such as with the castle or terrain or tetrahedron. I also used a melee hit detection for collision to then calculate damage and other variables. I finally also used a motion to allow the holy projectile from the priest to fall down and rotate whilst doing so and spawn multiple instances of it. (**Appendix 2** can be seen demonstrating distance checking combined with a dot product to ensure the player is always within the enemy's attack arc)
 - NPC/AI characters- Bezerker detects, chases and looks at the character and melees him. Priest detects and reacts to player and moves to a safe spot (see **Appendix 5**) and reacts appropriately and summons projectiles of holy halberds, the boss detects, chases, looks at the player and attacks him with a stomp. The 4th one is the holy halberd as it is a kinematic AI with its own autonomous movement and collision detection and reaction which in this case is its deletion on collision. Finally, the system uses 5 main states to manage transitions for the player. (See **Appendix 4** for the souls like dash/roll/dodge mechanic being implemented)
 - Resource management- The player has HP, Stamina and potion counts. Uses timers for attack and roll cooldowns and enemy state transitions. Upgrades are available too as souls can be used as a currency resource in the game after picking them up. (**Appendix 1** and **Appendix 8** can be seen demonstrating resource management and the usage of souls in the pause upgrade menu)

Overall, I achieved a lot in this project and was successfully able to make a well developed game as it covers all those features and a good overall theme.

Asset/Library Listings:

- External Mesh Assets:
 - Player (Knight) Animated Mesh:
 - Source URL: <https://opengameart.org/content/animated-knight>
 - Author(s): Clement Wu, Nikolaus & Botanic (Submitted by Botanic)
 - License(s): CC BY-SA 3.0, CC BY 3.0, GPL 3.0, GPL 2.0
 - Devil Boss (Diablous) Mesh:
 - Source URL: <https://opengameart.org/content/animated-diablous>
 - Author(s): Clement Wu, Nikolaus & Botanic (Submitted by Botanic)
 - License(s): CC BY-SA 3.0, CC BY 3.0, GPL 3.0, GPL 2.0
 - Bezerker
 - Source URL: <https://opengameart.org/content/animated-bezerker>
 - Author(s): Clement Wu, Nikolaus & Botanic (Submitted by Botanic)
 - License(s): CC BY-SA 3.0, CC BY 3.0, GPL 3.0, GPL 2.0
 - Priest
 - Source URL: <https://opengameart.org/content/animated-priest>
 - Author(s): Clement Wu, Nikolaus & Botanic (Submitted by Botanic)
 - License(s): CC BY-SA 3.0, CC BY 3.0, GPL 3.0, GPL 2.0
 - Halberd
 - Source URL: <https://sketchfab.com/3d-models/halberds-e7b633a334ae4120a6073d32a5056b4e>

- Author(s): soidev
 - License(s): CC BY 4.0
 - Elm Tree from the original template code
- Texture Assets:
 - Skybox
 - Source URL: <https://opengameart.org/content/stormy-days-skybox>
 - Author(s): Jockum Skoglund (hipshot) (Submitted by arikel)
 - License(s): CC BY 3.0
 - Rocky Terrain
 - Source URL: https://polyhaven.com/a/rocky_terrain_02
 - Author(s): Amal Kumar
 - License(s): CC0
 - Castle Textures
 - Source URL: <https://opengameart.org/content/low-res-seam-less-rock-textures>
 - Author(s): GrEmlin
 - License(s): CC BY-SA 4.0, CC0
 - Soul Fabric
 - Source URL: <https://polyhaven.com/textures/fabric>
 - Author(s): Rico Cilliers, colormass
 - License(s): CC0
- Audio Assets:
 - Soliloquy.mp3
 - Source URL: <https://opengameart.org/content/soliloquy>
 - Author(s): Matthew Pablo
 - License(s): CC BY 3.0
 - Pickup.wav
 - Source URL: <https://opengameart.org/content/key-pickup>
 - Author(s): Vinrax
 - License(s): CC BY 3.0
 - Attack.wav
 - Source URL: <https://opengameart.org/content/fleshy-fight-sounds>
 - Author(s): will_leton
 - License(s): OGA BY 3.0
 - Dash.wav
 - Source URL: <https://opengameart.org/content/falling-body>
 - Author(s): remaxim
 - License(s): CC BY-SA 3.0, GPL 3.0, GPL 2.0
- Core Libraries:
 - Bullet Physics which handles all complex rigid body physics and collision world setup
 - FMOD which handles all audio management and playback (music, spatialized sounds)
 - Assimp used for loading all 3D meshes in
 - GLM for all vector/matrix transformations and rotations

Discussion/Challenges I faced

- I faced many challenges in this project. However, I had the most difficulty when it came to dealing with transformations with 3D meshes, I downloaded online since half the time they didn't appear to be working. In addition to this, their rotations and positions would always be weirdly off for example, I had to offset the height by 0.9f for it to level to the ground.
- I also had trouble with stamina and combat synchronisation. To fix this, I had to use Boolean flags and precise timer logic to synchronise the animations, sound effects and stamina consumption to the FSM state changes.
- Another challenge I faced was that for some weird reason my code refused to work whenever I tried rotating the halberd vertically for holy projectiles. So, I decided to try a different approach and code it to rotate on the X axis instead which seemed to work way better. (See **Appendix 9** to see how I coded its fall and spin rate)

Overall, I would say that there weren't too many difficulties. Most issues seemed to come with silly mistakes like incorrectly naming something or incorrectly using a texture file or a model file that doesn't exist or work.

In order for this to hypothetically become a proper game to release, it would still need a lot more features – especially if I wish to mimic a souls like game. I would have to set up safe points, more scenery, more types of enemies, different weapons, more stats than just movement speed and attack damage and all sorts of stuff.

Source Code Design and Quality

Key Design Patterns

The project utilises modular and scalable design patterns to manage complexity:

- Finite State Machines (FSMs): All primary entities (player, Enemy, Boss, Priest) implement an explicit FSM using C++ classes to manage behaviour, animation, and state transitions (e.g., Idle, Attacking, Rolling, Dead). This provides a clear logic for complex AI and character control.
- Modular Encapsulation: Game logic is effectively separated from the engine/rendering loop. High-level management (spawning, rendering, event handling) is contained within `example_layer`, while all entity-specific rules (movement, damage, healing, state checks) are fully encapsulated within their respective class files (e.g., `Player.cpp`, `Priest.cpp`).
- Engine Abstraction: Custom primitives (tetrahedron, SoulFragment) were created as distinct classes, separating the vertex and normal calculation from their use as game objects, which improves overall modularity.

Organisation and Style

The code sticks to consistent C++ standards whilst also keeping readable and understandable:

- Conventional naming was used throughout the project with enough description to distinguish everything clearly.

- The logic is also cleanly split between header (.h) files for declaration and source (.cpp) files for implementation, following standard C++ practice.
- Readability and Comments: The code is well-commented, particularly within FSMs and complex functions
- Use of Constants: Fixed animation indices (m_anim_idle, m_anim_walk, etc.) and game balance stats(e.g., stamina costs, health values) are declared as class members or constants.
- Overall, the design successfully supports the complex, action-RPG mechanics, demonstrating good object-oriented principles and an understandable reading style.

Appendices

```

1. if (e.key_code() == engine::key_codes::KEY_1)
2. {
3.     if (m_soul_count >= 5) {
4.         m_soul_count -= 5;
5.         m_player.increase_damage(10.0f);
6.     }
7. }
8.

```

Appendix 1: This demonstrates Souls being used as currency for permanent stat upgrades

```

1. // Check for damage window in the middle of the attack animation
2. if (m_attack_cooldown > 1.5f && m_attack_cooldown < 2.0f && !m_damage_dealt)
3. {
4.     glm::vec3 to_player = m_player_target->position() - m_object->position();
5.     float dist = glm::length(to_player);
6.     if (dist > 0.01f) to_player = glm::normalize(to_player);
7.
8.     float dot_prod = glm::dot(m_object->forward(), to_player);
9.
10.    //Check distance AND if the player is in front (dot product check)
11.    if (dist <= m_attack_range + 1.5f && dot_prod > 0.5f)
12.    {
13.        m_damage_dealt = true;
14.        m_damage_signal = true; // Signal to example_layer
15.    }
16. }
17.

```

Appendix 2: Collision is implemented using a physics-based distance check combined with a dot product

```

namespace engine
{
    // NOTE: The geometry created here is an octahedron (two pyramids base-to-base), which has 8 faces.
    SoulFragment::SoulFragment(std::vector<glm::vec3> vertices) : m_vertices(vertices)
    {
        // An octahedron has 8 triangle faces
        // We need 8 normals, one for each face
        std::vector<glm::vec3> normals;

        // top tetrahedron faces
        normals.push_back(glm::normalize(glm::cross(vertices.at(1) - vertices.at(0), vertices.at(2) - vertices.at(0)))); // Front-Right
        normals.push_back(glm::normalize(glm::cross(vertices.at(2) - vertices.at(0), vertices.at(3) - vertices.at(0)))); // Front-Left
        normals.push_back(glm::normalize(glm::cross(vertices.at(3) - vertices.at(0), vertices.at(4) - vertices.at(0)))); // Back-Left
        normals.push_back(glm::normalize(glm::cross(vertices.at(4) - vertices.at(0), vertices.at(1) - vertices.at(0)))); // Back-Right

        // bottom ones
        normals.push_back(glm::normalize(glm::cross(vertices.at(2) - vertices.at(5), vertices.at(1) - vertices.at(5)))); // Bottom Front-Right
        normals.push_back(glm::normalize(glm::cross(vertices.at(3) - vertices.at(5), vertices.at(2) - vertices.at(5)))); // Bottom Front-Left
        normals.push_back(glm::normalize(glm::cross(vertices.at(4) - vertices.at(5), vertices.at(3) - vertices.at(5)))); // Bottom Back-Left
        normals.push_back(glm::normalize(glm::cross(vertices.at(1) - vertices.at(5), vertices.at(4) - vertices.at(5)))); // Bottom Back-Right

        // An octahedron has 24 vertices in total (8 faces * 3 vertices/face)
    }
}

```

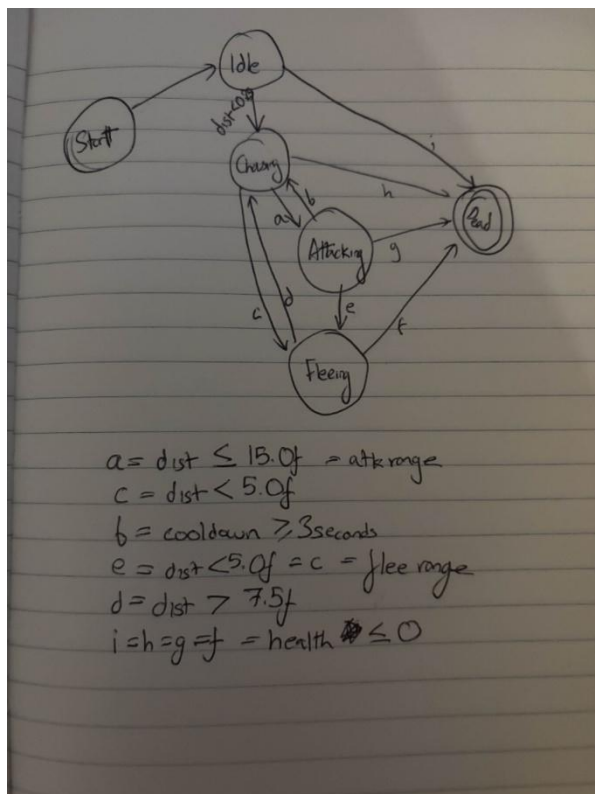
Appendix 3: normals.push_back(glm::normalize(glm::cross(vertices.at(1) - vertices.at(0), vertices.at(2) - vertices.at(0)))); // Front-Right (The normal vector for this triangular face is manually calculated using the cross product of two edge vectors)

```

1. //ROLLING STATE (Invulnerability)
2. else if (m_current_state == PlayerState::Rolling)
3. {
4.     m_animation_timer -= (float)time_step;
5.
6.     float dash_speed = m_run_speed * 3.0f;
7.     glm::vec3 current_vel = m_object->velocity();
8.     //Applies a constant velocity in the locked roll direction just like in every souls game
9.     m_object->set_velocity(glm::vec3(m_locked_roll_direction.x * dash_speed, current_vel.y,
m_locked_roll_direction.z * dash_speed));
10.
11.     m_object->set_rotation_axis(glm::vec3(0.f, 1.f, 0.f));
12.     m_object->set_rotation_amount(m_dash_angle);
13.
14.     m_object->set_offset(glm::vec3(0.f, -9.0f, 0.f));
15.     m_object->animated_mesh()->on_update(time_step * 3.0f); //Speed up its animation
16.
17.     if (m_animation_timer < 0.0f)
18.     {
19.         //Exit rolling state
20.         m_object->set_velocity(glm::vec3(0.f, m_object->velocity().y, 0.f));
21.         m_object->animated_mesh()->switch_root_movement(false);
22.         m_object->animated_mesh()->switch_animation(m_anim_idle);
23.         m_current_state = PlayerState::Idle;
24.     }
25. }
26.

```

Appendix 4: Invulnerability dark souls type dodge roll dash action successfully implemented as a game mechanic



Appendix 5: State machine diagram for the Priest AI

Controls:
WASD: Move | L-Shift: Dash | R: Health Potion
Mouse: Move Camera
Space: Attack | E: Toggle Upgrade Menu
P: Spawn Enemy

medievalsoulsproject

START

SENSITIVITY < 0.50 >

> VOLUME < 0.05 >

Appendix 6: Main menu screenshot



Appendix 7: Holographic spinning halberd projectile spawned by the priest mage



Appendix 8: The pause upgrade menu showing resource management and the ability to pause the game

```

1. void HolyProjectile::on_update(const engine::timestep& time_step)
2. {
3.     if (!m_active) return;
4.
5.     float dt = (float)time_step;
6.
7.     //Fall down
8.     glm::vec3 pos = m_object->position();
9.     pos.y -= m_speed * dt;
10.    m_object->set_position(pos);
11.    //Spin amount
12.    float current_rot = m_object->rotation_amount();
13.    m_object->set_rotation_amount(current_rot + 3.0f * dt);
14.    //Deactivates if it touches the floor
15.    if (pos.y <= 0.0f) {
16.        m_active = false;
17.    }
18. }
19.

```

Appendix 9: Holy Projectile's custom movement